Assignment 2

Aayush Anand
Roll No. 230025

Setup

All measurements are on GPU3: NVIDIA A40 (compute capability 8.6, 84 SMs, 46 GB, 6 MB L2), CUDA 12.6, gcc 11.4, Intel Xeon Silver 4208. Another job was using the GPU while I measured, so very short kernels vary a bit between runs.

Running `make` in the top folder builds everything with:

```
ARCH = -gencode arch=compute_86,code=sm_86
nvcc -O2 -std=c++17 $(ARCH) problem1/230025-prob1.cu -o problem1/problem1.out
gcc  -O2 -std=c17 -D_POSIX_C_SOURCE=199309L problem2/problem2-v0.c -o problem2/problem2-v0.out -lm
nvcc -O2 -std=c++17 $(ARCH) -fmad=false problem2/230025-prob2-v1.cu -o problem2/problem2-v1.out
nvcc -O2 -std=c++17 $(ARCH) -fmad=false problem2/230025-prob2-v2.cu -o problem2/problem2-v2.out
nvcc -O2 -std=c++17 $(ARCH) -fmad=false problem2/230025-prob2-v3.cu -o problem2/problem2-v3.out
nvcc -O2 -std=c++17 $(ARCH) problem3/230025-prob3-vanilla.cu -o problem3/problem3-vanilla.out
nvcc -O2 -std=c++17 $(ARCH) problem3/230025-prob3-blocked.cu -o problem3/problem3-blocked.out
g++  -O2 -std=c++17 problem3/gen_traces.cpp -o problem3/gen_traces.out
```

Running:

```
./problem1/problem1.out            # N = 64, 128, 256 (or pass one N)
cd problem2; ./problem2-v1.out     # reads ./disp.txt, ./grid.txt, writes results-v1.txt
./problem3/problem3-vanilla.out N <insert-trace> <search-trace> <delete-trace>
./problem3/problem3-blocked.out N <insert-trace> <search-trace> <delete-trace>
```

1 Problem 1: 3D convolution

Each output is the average of its $5 \times 5 \times 5$ neighbourhood weighted by the filter, with ghost cells taken as 0:

$$\text{out}[z][y][x] = \frac{1}{125} \sum_{f_z, f_y, f_x = 0}^{4} F[f_z][f_y][f_x] \cdot \text{in}[z + f_z - 2][y + f_y - 2][x + f_x - 2].$$

The input is filled with random values in $[0, 1)$.

- **Basic kernel**: one thread per output, $8 \times 8 \times 8$ blocks, input and filter read from global memory.
- **Optimized kernel**:
  (a) *Shared memory tiling*: a 32×8 thread block computes a $32 \times 8 \times 8$ output tile (8 outputs per thread along z). The block first loads the $36 \times 12 \times 12$ input tile with its halo into shared memory, writing 0 for ghost cells, so the compute loop needs no bounds checks.
  (b) *Constant memory*: the filter is in `__constant__` memory. All threads of a warp read the same filter element at the same time, so it is a broadcast.

(c) *Loop unrolling*: the f_z, f_y, f_x loops and the loop over the 8 outputs are fully unrolled with `#pragma unroll`.

`check_result()` reports no differences for both kernels at all three sizes. The CPU and the kernels use `fmaf` and add the terms in the same order; otherwise nvcc's fused multiply-add rounds differently from the CPU and the `FLT_EPSILON` threshold of `check_result()` fails.

Kernel time is measured with CUDA events. End-to-end time is the copy of the input (and filter) to the GPU, the kernel, and the copy of the output back. Both are averaged over 20 runs (after 3 warm-up runs), and the table shows the mean of 3 such program runs.

		Kernel (ms)			End-to-end (ms)		
N	CPU (ms)	basic	opt	speedup	basic	opt	speedup
64	124	0.0444	0.0162	2.7×	1.08	1.00	1.07×
128	823	0.293	0.0713	4.1×	4.87	4.26	1.14×
256	6772	3.11	0.493	6.3×	36.7	31.4	1.17×

Table 1: Problem 1 on the A40.

The optimized kernel is up to 6.3× faster. The basic kernel does 125 global loads per output, while the tiled kernel loads each input element about 2.5 times and then works out of shared memory. The gain is smaller at $N = 64$ because the grid is tiny (128 blocks for 84 SMs). End-to-end time is dominated by the PCIe transfers (64 MB each way at $N = 256$), so the kernel speedup hardly shows there.

2 Problem 2: 10D grid search

Every loop runs 13 times, so there are $13^{10} \approx 1.38 \times 10^{11}$ points. The sequential code finds 11608 points in 689.3 s.

Correctness. All three versions write `results-vX.txt` files that are byte for byte identical to `results-v0.txt` (checked with `cmp`). For this, the CUDA code is compiled with `-fmad=false`, so no multiply-add is fused into an FMA (gcc does not fuse either), and every accepted point is evaluated with the same double expression in the same order as the C code. The kernels return the linear index of each valid point; the host sorts the indices, which gives the loop order, and prints the points the same way as the C code.

Version 1. One thread per point. Each launch covers the innermost 7 loops (13^7 points), so there are 2197 launches. A thread decodes its linear index into $r_1..r_{10}$, evaluates the 10 constraints exactly as the C code, and appends the index of a valid point to a device buffer using `atomicAdd` on a counter. After each launch the host copies the new results back (the buffer is enlarged and the chunk rerun if it overflows).

Version 2.

- *More work per thread*: a thread handles a prefix $(r_1..r_7)$ and runs the r_8, r_9, r_{10} loops itself, keeping the partial sum of every constraint so each loop level adds only one term.
- *Float filter, exact double check*: the partial sums are kept in float (FP32 is 64× faster than FP64 on the A40). They are used only to reject points, with a safety margin of 10^{-5} of the size of

the terms, which is much larger than any float rounding error. Points that pass are rechecked in double exactly as in the C code.

- *Pruning*: for every level the host precomputes the minimum and maximum the remaining terms can add. A prefix is skipped when no completion can bring some constraint within its tolerance.
- *Shared memory tiling*: the products $c_{ji}\, x_i(r)$ for all dimensions and values, and the bound tables (about 6 KB), are loaded into shared memory once per block; blocks then loop over many prefixes. The parameters are in constant memory.
- *Multiple kernels and streams*: one launch per value of r_1, alternating between two streams, all writing to one output buffer, with a single copy back at the end.

Version 3 (Thrust). The loops are split into two halves, $A = (r_1..r_5)$ and $B = (r_6..r_{10})$, with 13^5 entries each. For one constraint the value is $P^A + P^B$, so after sorting B by P^B the Bs that can match a given A form one contiguous range. Thrust calls used: `transform` over `counting_iterator` (partial sums), `sequence` + `sort_by_key` (sort B), `lower_bound`/`upper_bound` with a `transform_iterator` (range for every A), `exclusive_scan` and `reduce` (offsets and counts), `upper_bound` + `transform` (expand candidate pairs), `copy_if` with the exact double check (keep valid points) and `sort` (output order). The constraint giving the fewest candidate pairs is chosen at run time; here it is constraint 1 with 45.4 million candidates.

Version	GPU time (s)	Total time (s)	Speedup over v0
v0 (sequential CPU)		689.3	1×
v1	203.4	204.6	3.4×
v2	0.014	0.060	11,500×
v3 (Thrust)	0.34	0.39	1,800×

Table 2: Problem 2 on the A40. GPU time: CUDA events around all kernels (all Thrust calls for v3). Total: from after reading the input to after writing the results file, as in the C code. v3 is the median of 8 runs.

v1 is limited by double precision throughput: every point needs about 220 FP64 operations, and the A40 runs FP64 at 1/64 of the FP32 rate. v2 is the fastest because it does far less work: one float addition per loop level instead of a full double evaluation per point, and whole subtrees are skipped by pruning. Its total time is mostly writing the results file. v3 avoids most of the space too, but still checks 45 million candidate pairs in double and runs many small Thrust kernels, so it is slower than v2.

3 Problem 3: Counting Bloom filter

Design.

- *Counters*: a raw array of N 8-bit counters, four packed in each 32-bit word (counter p is byte p mod 4 of word $p/4$). Since there are no 8-bit atomics, `insert()` does `atomicAdd(&word, 1 << 8*(p%4))` (overflow ignored), and `delete()` uses an `atomicCAS` loop on the word that decrements the byte only if it is non-zero. `search()` reads the bytes and stops at the first zero counter. One thread per key.
- *Hash functions*: `hash1()`..`hash4()` mix the key with different constants, apply the MurmurHash3 32-bit finalizer, and map to $[0, n)$ with `(h * n) >> 32`. The vanilla filter uses `hash1`..`hash3` over

$[0, N)$.

- *Blocked variant*: `hash1` selects a block of 128 counters (128 bytes, one L1 line), and `hash2`, `hash3` and `hash4` select the three counters inside it.
- *Duplicates*: before insert and delete, a dedup kernel keeps one copy of each key using a raw open addressing table and `atomicCAS`, so repeated keys do not increment the counters again.
- *Delete*: `search()` is first run on the batch, and `delete()` only decrements keys that were found, so deleting a key that is not present does nothing.

check_result(). The host replays every operation on a sequential copy of the filter (same hashes and layout) and keeps the real key set in a `std::unordered_set`. After each operation all GPU counters must match the host copy, and every search result must match the host lookup. The real set gives the false positives. False negatives are reported as an error unless they come from deleting a key that was never inserted but looked present. A counting Bloom filter cannot avoid those.

Traces. `gen_traces.cpp` generates them. Included: `small` (10^4 inserts) and `medium` (10^6 inserts and searches, 5×10^5 deletes); about 10% of the inserts are repeats, half of the searches are for inserted keys, and 80% of the deletes are for inserted keys. The large trace (10^7 keys) is created with `./gen_traces.out 10000000 10000000 5000000 large 3`.

Results. Both variants pass `check_result()` on all traces. False positive rate (false positives / all searches):

		Before delete		After delete	
Trace	N	vanilla	blocked	vanilla	blocked
small	10^5	0.00680	0.00560	0.00300	0.00280
medium	2^{23}	0.01028	0.01183	0.00475	0.00605
large	2^{27}	0.00294	0.00386	0.00129	0.00192

Table 3: False positive rates.

The blocked filter has a higher false positive rate (15% higher on medium, 31% on large) because keys do not spread evenly across blocks, so some blocks fill up more than others. On the small trace the difference is within noise.

Profiling. Nsight Compute on the large trace, $N = 2^{27}$:

```
ncu --set full -k regex:"insert_kernel|search_kernel" --launch-skip 2 --launch-count 2 \
    ./problem3-vanilla.out 134217728 large-insert.bin large-search.bin large-delete.bin
```

	vanilla		blocked	
Metric	insert	search	insert	search
Duration (ms)	9.42	3.17	5.02	3.01
DRAM throughput (% of peak)	52.6	62.8	66.9	60.0
Memory throughput (GB/s)	366	437	465	417
Arithmetic intensity (FLOP/byte)	0	0	0	0
Achieved occupancy (%)	68.4	87.2	70.2	74.5
Warp efficiency (active threads per warp)	32.0	24.1	32.0	29.7
L2 hit rate (%)	1.17	2.80	23.6	6.95
Long scoreboard stall (cycles/instruction)	576.8	862.4	238.1	553.6

Table 4: Nsight Compute metrics on the A40.

The kernels do no floating point work, so the roofline arithmetic intensity is 0 (about 0.2 to 0.6 integer instructions per DRAM byte); they are memory bound. The 128 MB filter is much larger than the L2, so almost every access goes to DRAM. In the blocked insert, the three atomics of a key hit the same line, which raises the L2 hit rate to 23.6% and makes it 1.9× faster.

Top three bottlenecks reported by the profiler (same for both variants):

1. **Uncoalesced global accesses**: 84% (insert) and 89% (search) of the L2 sectors are excessive, because hashing sends the threads of a warp to unrelated cache lines.
2. **Long scoreboard stalls**: warps wait on global loads, 862 cycles per instruction in the vanilla search (97% of its stall time).
3. **Atomics throttling the memory pipeline** (insert): large LG throttle, MIO throttle and drain stalls (about 500 cycles each in vanilla) as the queues fill with atomics. Achieved occupancy is also only about 70%.